



Casa abierta al tiempo

UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Iztapalapa

Practica 2

UEA: Programación Concurrente

Profesor: Dr. Benjamín Moreno Montiel

Alumno: Soria Rodríguez Juan Antonio

Matricula: 2203041232

Fecha de Entrega: viernes 08 de noviembre del
2024

Documentación para el Programa:

Practica1_A.c

Programa: Métodos de Ordenamiento

Introducción

En la ciencia de la computación y en la programación, el ordenamiento de datos es una de las tareas fundamentales que permiten la organización y clasificación eficiente de la información. Existen múltiples algoritmos de ordenamiento que varían en términos de eficiencia y complejidad. El presente programa, Practica1_A.c, está diseñado para implementar y comparar varios métodos de ordenamiento, proporcionando un entorno interactivo para el usuario. Los algoritmos de ordenamiento incluidos son: Burbuja, Inserción Directa, Selección Directa y Merge Sort Recursivo.

Desarrollo

El programa está escrito en lenguaje C y se estructura en varias funciones clave para manejar diferentes aspectos de la ordenación y la interacción con el usuario:

- **Definición y Prototipos de Funciones:**

- burbuja(int n): Implementa el algoritmo de ordenamiento Burbuja.
- insercionDirecta(int n): Implementa el algoritmo de Inserción Directa.
- seleccionDirecta(int n): Implementa el algoritmo de Selección Directa.
- mergeSort(int ini, int fin): Implementa el algoritmo Merge Sort recursivo.
- merge(int ini, int mitad, int fin): Función auxiliar para mezclar dos subarreglos en Merge Sort.
- imprimeArreglo(int n): Imprime los elementos del arreglo.
- llenarArreglo(int n): Llena el arreglo con valores aleatorios.
- esperarTecla(void): Espera a que el usuario presione una tecla para continuar.

- **Función main():**

- Presenta un menú interactivo con opciones para seleccionar y ejecutar los distintos métodos de ordenamiento.
- Solicita al usuario el tamaño del arreglo y si desea visualizar los elementos antes y después del ordenamiento.
- Asigna memoria dinámica para los arreglos de trabajo.
- Llena el arreglo con valores aleatorios.

- Ejecuta el algoritmo de ordenamiento seleccionado, midiendo y mostrando el tiempo de ejecución.
- **Funciones de Ordenamiento:**
 - Cada función de ordenamiento implementa su respectivo algoritmo, recorriendo y modificando el arreglo según las reglas del método correspondiente.
- **Interactividad y Gestión de Memoria:**
 - El programa permite al usuario seleccionar entre diferentes algoritmos y visualizar los resultados.
 - Gestiona de manera eficiente la memoria asignada dinámicamente para asegurar que no haya fugas de memoria.

Conclusión general

El programa Practica1_A.c proporciona una herramienta interactiva que permite entender y comparar varios métodos de ordenamiento de manera práctica. Al implementar los algoritmos de Burbuja, Inserción Directa, Selección Directa y Merge Sort, se obtiene una comprensión más profunda de cómo funcionan estos métodos y cuál es su rendimiento en diferentes escenarios. Este ejercicio es fundamental para el aprendizaje de técnicas de programación y algoritmos, contribuyendo significativamente al desarrollo de habilidades en la gestión y manipulación de datos.

Conclusión Individual de Cada Método

- **Burbuja:** Mejor evitarlo para grandes arreglos debido a su ineficiencia, pero útil como herramienta educativa para entender conceptos básicos de ordenamiento.
- **Inserción Directa:** Eficiente para arreglos pequeños o casi ordenados; sin embargo, no es recomendable para arreglos desordenados de gran tamaño.
- **Selección Directa:** También sencillo y educativo, pero su eficiencia es comparable a la del método Burbuja, haciéndolo menos atractivo para grandes volúmenes de datos.
- **Merge Sort:** Destaca por su eficiencia y es ideal para grandes volúmenes de datos, aunque requiere espacio adicional.

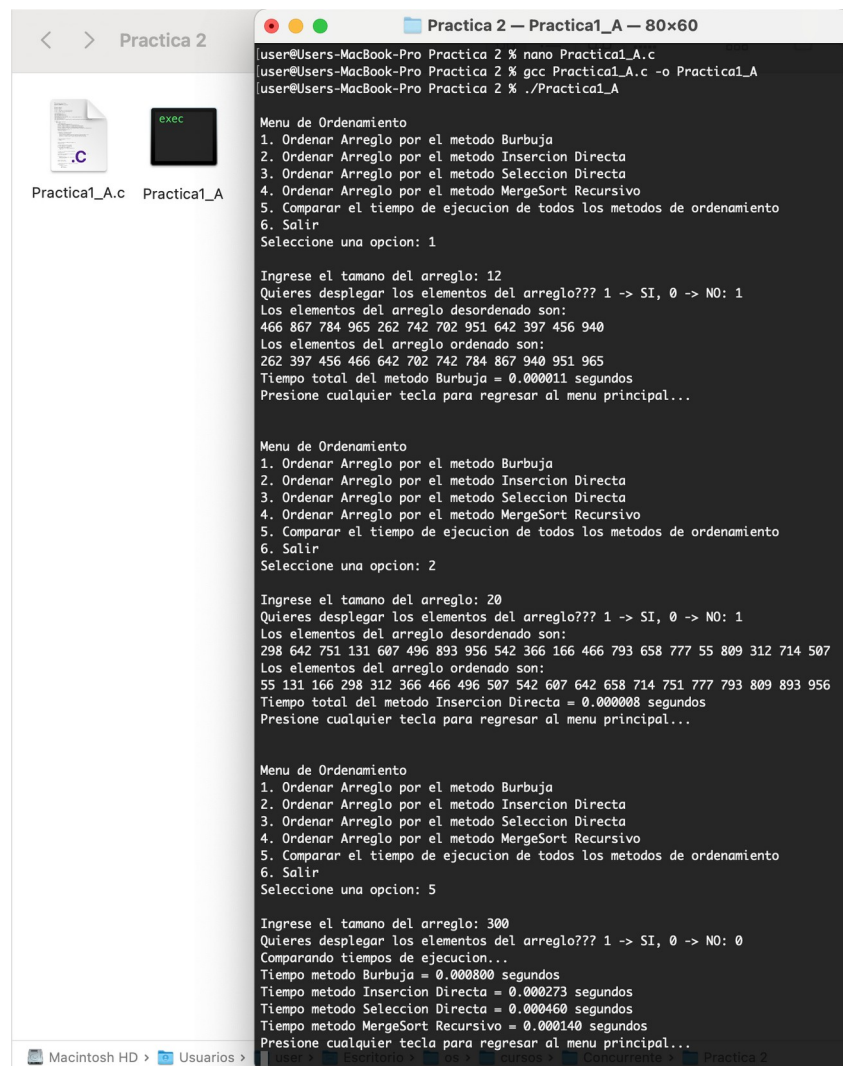
Comparación

1. Mejor Método: Merge Sort.

- **Razonamiento:** Debido a su complejidad temporal de $O(n \log n)$ y su eficiencia independientemente del estado inicial del arreglo, Merge Sort es el más adecuado para manejar grandes volúmenes de datos. Además, su capacidad para dividir y conquistar hace que sea robusto y confiable.

2. Peor Método: Burbuja.

- **Razonamiento:** Aunque es fácil de implementar y entender, el método Burbuja es el más ineficiente en términos de complejidad temporal ($O(n^2)$) en todos los casos. Esto lo hace inadecuado para arreglos grandes, donde su rendimiento es significativamente inferior en comparación con los otros métodos.
- **Ejecución:**



```
user@Users-MacBook-Pro Practica 2 % nano Practica1_A.c
user@Users-MacBook-Pro Practica 2 % gcc Practica1_A.c -o Practica1_A
user@Users-MacBook-Pro Practica 2 % ./Practica1_A

Menu de Ordenamiento
1. Ordenar Arreglo por el metodo Burbuja
2. Ordenar Arreglo por el metodo Insercion Directa
3. Ordenar Arreglo por el metodo Seleccion Directa
4. Ordenar Arreglo por el metodo MergeSort Recursivo
5. Comparar el tiempo de ejecucion de todos los metodos de ordenamiento
6. Salir
Seleccione una opcion: 1

Ingrese el tamano del arreglo: 12
Quieres desplegar los elementos del arreglo??? 1 -> SI, 0 -> NO: 1
Los elementos del arreglo desordenado son:
466 867 784 965 262 742 702 951 642 397 456 940
Los elementos del arreglo ordenado son:
262 397 456 466 642 702 742 784 867 940 951 965
Tiempo total del metodo Burbuja = 0.000011 segundos
Presione cualquier tecla para regresar al menu principal...

Menu de Ordenamiento
1. Ordenar Arreglo por el metodo Burbuja
2. Ordenar Arreglo por el metodo Insercion Directa
3. Ordenar Arreglo por el metodo Seleccion Directa
4. Ordenar Arreglo por el metodo MergeSort Recursivo
5. Comparar el tiempo de ejecucion de todos los metodos de ordenamiento
6. Salir
Seleccione una opcion: 2

Ingrese el tamano del arreglo: 20
Quieres desplegar los elementos del arreglo??? 1 -> SI, 0 -> NO: 1
Los elementos del arreglo desordenado son:
298 642 751 131 607 496 893 956 542 366 166 466 793 658 777 55 809 312 714 507
Los elementos del arreglo ordenado son:
55 131 166 298 312 366 466 496 507 542 607 642 658 714 751 777 793 809 893 956
Tiempo total del metodo Insercion Directa = 0.000008 segundos
Presione cualquier tecla para regresar al menu principal...

Menu de Ordenamiento
1. Ordenar Arreglo por el metodo Burbuja
2. Ordenar Arreglo por el metodo Insercion Directa
3. Ordenar Arreglo por el metodo Seleccion Directa
4. Ordenar Arreglo por el metodo MergeSort Recursivo
5. Comparar el tiempo de ejecucion de todos los metodos de ordenamiento
6. Salir
Seleccione una opcion: 5

Ingrese el tamano del arreglo: 300
Quieres desplegar los elementos del arreglo??? 1 -> SI, 0 -> NO: 0
Comparando tiempos de ejecucion...
Tiempo metodo Burbuja = 0.000800 segundos
Tiempo metodo Insercion Directa = 0.000273 segundos
Tiempo metodo Seleccion Directa = 0.000460 segundos
Tiempo metodo MergeSort Recursivo = 0.000140 segundos
Presione cualquier tecla para regresar al menu principal...
```

Documentación para el Programa **Practica1_B.c**

Programa: Listas Simples Ligadas

Introducción

En la programación, las estructuras de datos son fundamentales para la organización y gestión eficiente de la información. Una de las estructuras de datos más versátiles y comunes es la lista enlazada. A diferencia de los arreglos estáticos, las listas enlazadas permiten la inserción y eliminación dinámica de elementos sin la necesidad de reorganizar la memoria. El presente programa, **Practica1_B.c**, implementa una lista simplemente ligada, proporcionando una interfaz interactiva para realizar varias operaciones, como insertar, eliminar, consultar y buscar elementos en la lista.

Desarrollo

El programa está escrito en lenguaje C y se estructura en varias funciones clave para manejar la lista simplemente ligada y la interacción con el usuario:

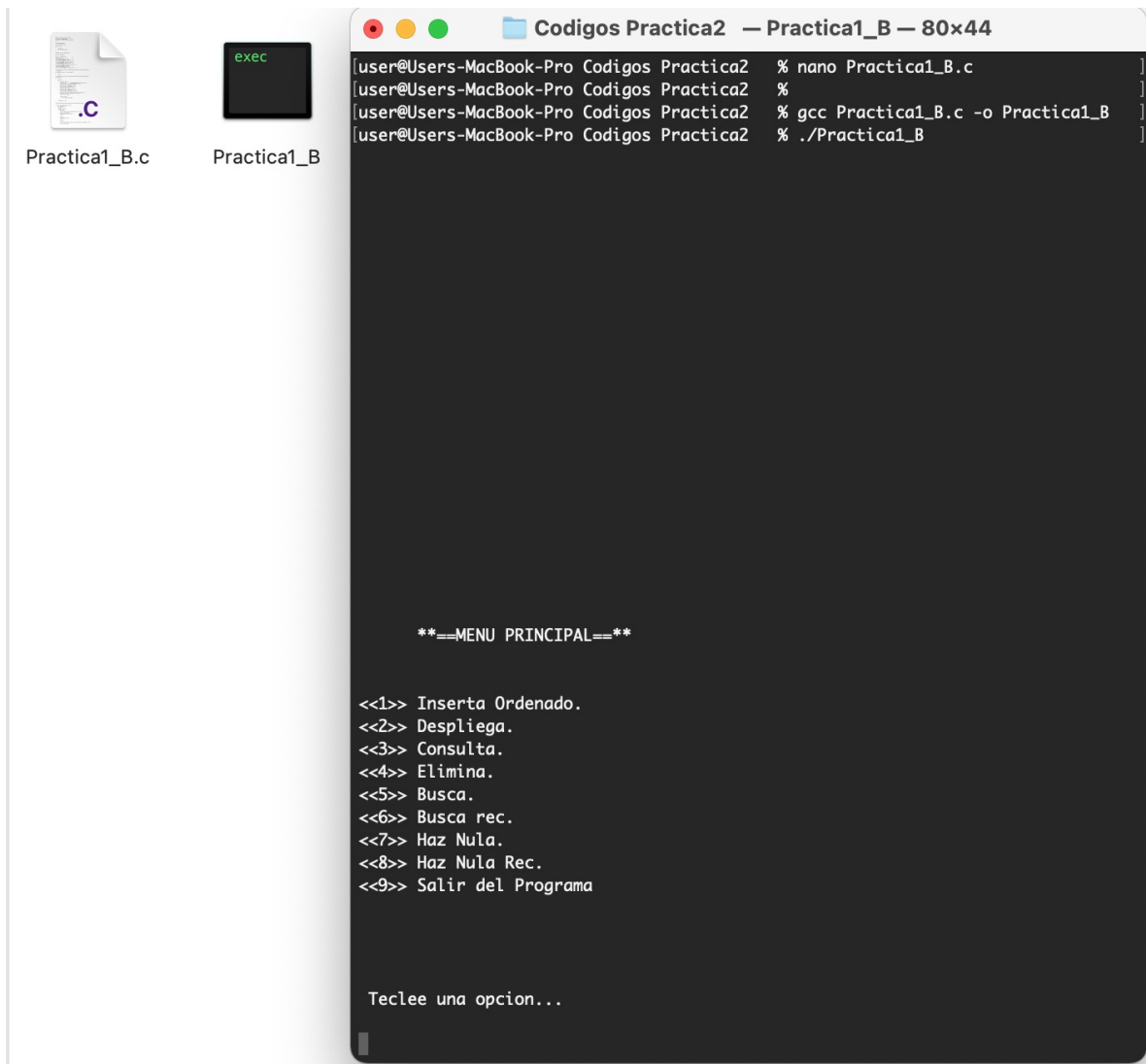
- **Definición de la Estructura nodo:**
 - La estructura `nodo` contiene un entero `info` y un puntero `sig` que apunta al siguiente nodo en la lista enlazada.
 - Se define un tipo `Nodo` usando `typedef` para facilitar el manejo de nodos.
- **Declaración de Funciones:**
 - `creanodo(int valor)`: Crea un nuevo nodo con un valor entero.
 - `Menu()`: Muestra el menú de opciones y solicita al usuario que seleccione una opción.
 - `EjecutarOpcion(int opc)`: Ejecuta la opción seleccionada del menú.
 - `despliega(Nodo *lista)`: Muestra los elementos de la lista enlazada.
 - `consulta(Nodo *lista, int pos)`: Consulta el valor en una posición específica de la lista.
 - `elimina(Nodo **lista, int valor)`: Elimina un nodo con un valor específico de la lista.
 - `busca(Nodo *lista, int valor)`: Busca un nodo con un valor específico en la lista.
 - `buscarec(Nodo *lista, int valor, int index)`: Busca recursivamente un nodo con un valor específico en la lista.
 - `haznula(Nodo **lista)`: Vacía la lista.
 - `hnrec(Nodo **lista)`: Vacía recursivamente la lista.

- `insertaord(Nodo **lista, int valor)`: Inserta un nuevo nodo en orden ascendente en la lista.
- **Función `main()`:**
 - Llama a la función `Menu()` para iniciar la interacción con el usuario.
- **Funciones de Manipulación de la Lista:**
 - `creanodo`: Crea un nuevo nodo asignando memoria dinámica.
 - `insertaord`: Inserta un nuevo nodo en la lista en orden ascendente.
 - `despliega`: Muestra los elementos de la lista enlazada de forma ordenada.
 - `consulta`: Verifica el valor en una posición específica de la lista.
 - `elimina`: Elimina un nodo de la lista si encuentra el valor especificado.
 - `busca`: Busca un valor en la lista y devuelve su posición.
 - `buscarec`: Realiza la búsqueda de un valor en la lista de manera recursiva.
 - `haznulla`: Vacía la lista eliminando todos sus elementos.
 - `hnrec`: Vacía la lista de manera recursiva, liberando la memoria de cada nodo.

Conclusión

El programa `Practica1_B.c` proporciona una implementación completa y funcional de una lista simplemente ligada, permitiendo al usuario interactuar y realizar diversas operaciones sobre la lista a través de un menú interactivo. Esta práctica demuestra la importancia y utilidad de las listas enlazadas en la programación, especialmente cuando se requiere flexibilidad en la inserción y eliminación de elementos. Cada función está diseñada para manejar un aspecto específico de la lista, asegurando una gestión eficiente de la memoria y una experiencia de usuario fluida. Este ejercicio es esencial para entender las operaciones básicas de las listas enlazadas y su aplicación en problemas más complejos de la computación.

- **Ejecución:**



```
Codigos Practica2 — Practica1_B — 80x44

<<1>> Inserta Ordenado.
<<2>> Despliega.
<<3>> Consulta.
<<4>> Elimina.
<<5>> Busca.
<<6>> Busca rec.
<<7>> Haz Nula.
<<8>> Haz Nula Rec.
<<9>> Salir del Programa

Teclee una opcion...

2

[3]->[5]->[8]->NULL

Presione 'enter' para continuar: . . . █
```



```
<<5>> Busca.  
<<6>> Busca rec.  
<<7>> Haz Nula.  
<<8>> Haz Nula Rec.  
<<9>> Salir del Programa
```

Teclee una opcion...

6

Escribe el elemento que quieres buscar:

8

El elemento en la posicion 8 se encuentra en la poscicion 2 . . .

Presione 'enter' para continuar: . . . █